



Information Systems for Big Data – 12th Lesson

Prof. G. Fenza gfenza@unisa.it



Spark MLlib

Introduction



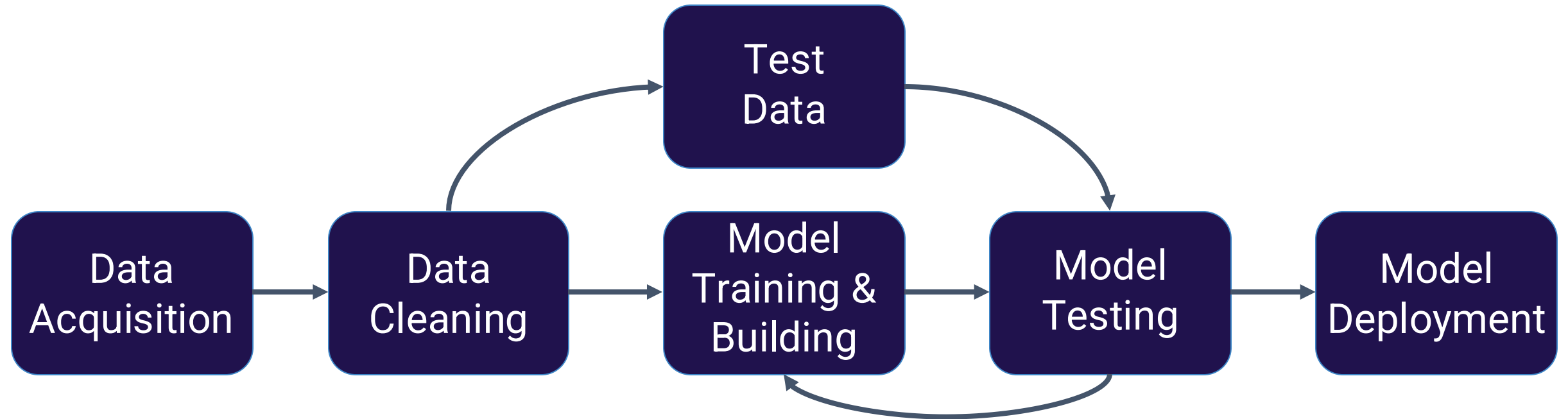
- Machine learning is a method of data analysis that automates analytical model building.
- Using algorithms that iteratively learn from data, machine learning allows computers to find hidden insights without being explicitly programmed where to look.

Machine Learning Applications



- Fraud detection.
- Web search results.
- Real-time ads on web pages
- Credit scoring and next-best offers.
- Prediction of equipment failures.
- New pricing models.
- Network intrusion detection.
- Recommendation Engines
- Customer Segmentation
- Text Sentiment Analysis
- Predicting Customer Churn
- Pattern and image recognition.
- Email spam filtering.
- Financial Modeling
- etc.

Machine Learning Process



Supervised vs Unsupervised



- **Supervised learning** algorithms are trained using **labeled** examples, such as an input where the desired output is known. For example, a piece of equipment could have data points labeled either “F” (failed) or “R” (runs).
- Methods: classification, regression, gradient boosting, supervised learning uses patterns to predict the values of the label on additional unlabeled data.
- For example,
 - It can anticipate when credit card transactions are likely to be fraudulent or which insurance customer is likely to file a claim (Classification);
 - It can attempt to predict the price of a house based on different features for houses for which we have historical price data (Regression).

Supervised vs Unsupervised



- **Unsupervised learning** is used against data that has no historical labels.
- The system is not told the "right answer." The algorithm must figure out what is being shown.
- The goal is to explore the data and find some structure within.
- For example, it can find the main attributes that separate customer segments from each other.
- Popular techniques include self-organizing maps, nearest-neighbor mapping, k-means clustering and singular value decomposition.
- **One issue is that it can be difficult to evaluate results of an unsupervised model!**

<https://spark.apache.org/docs/latest/ml-guide.html>

- ML Algorithms: common learning algorithms such as classification, regression, clustering, and collaborative filtering
- Featurization: feature extraction, transformation, dimensionality reduction, and selection
- Pipelines: tools for constructing, evaluating, and tuning ML Pipelines
- Persistence: saving and load algorithms, models, and Pipelines
- Utilities: linear algebra, statistics, data handling, etc.



SparkML

Regression Algorithm Example

Linear Regression – Hands on PySpark



- Start spark session
- File: **sample_linear_regression_data.txt**
- Import: **LinearRegression**

```
from pyspark.ml.regression import LinearRegression
```

```
# Load training data  
training = spark.read.format("libsvm").load("./sample_linear_regression_dat
```



label	features
-9.490009878824548	(10, [0, 1, 2, 3, 4, 5, ...]

LibSVM format



- Label
- #Feature:#Corresponding_NonNull_Value

File: **sample_linear_regression_data.txt**

```
1 -9.490009878824548 1:0.4551273600657362 2:0.36644694351969087 3:-0.38256108933468047
  4:-0.4458430198517267 5:0.33109790358914726 6:0.8067445293443565 7:-0.2624341731773887
  8:-0.44850386111659524 9:-0.07269284838169332 10:0.5658035575800715
2 0.2577820163584905 1:0.8386555657374337 2:-0.1270180511534269 3:0.499812362510895 4:-0.22686625128130267
  5:-0.6452430441812433 6:0.18869982177936828 7:-0.5804648622673358 8:0.651931743775642
  9:-0.6555641246242951 10:0.17485476357259122
3 -4.438869807456516 1:0.5025608135349202 2:0.14208069682973434 3:0.16004976900412138 4:0.505019897181302
  5:-0.9371635223468384 6:-0.2841601610457427 7:0.6355938616712786 8:-0.1646249064941625
  9:0.9480713629917628 10:0.42681251564645817
4 -19.782762789614537 1:-0.0388509668871313 2:-0.4166870051763918 3:0.8997202693189332
  4:0.6409836467726933 5:0.273289095712564 6:-0.26175701211620517 7:-0.2794902492677298
  8:-0.1306778297187794 9:-0.08536581111046115 10:-0.05462315824828923
5 -7.966593841555266 1:-0.06195495876886281 2:0.6546448480299902 3:-0.6979368909424835
  4:0.6677324708883314 5:-0.07938725467767771 6:-0.43885601665437957 7:-0.608071585153688
  8:-0.6414531182501653 9:0.7313735926547045 10:-0.026818676347611925
```

LibSVM format



- LibSVM allows for sparse training data.
 - That is, the non-zero values are the only ones that are included in the dataset. Hence, the index specifies the column of the instance data (feature index).
 - Label
 - ❖ Numerical label: matching classification group, or regression numerical value.
 - Features
 - ❖ Vector: all the features that belong to that row.
- With Spark
 - Usually what we end up doing is combining the various feature columns we have into a single 'features' column using the data transformations in Spark
 - Transformations allow us to carry out the required data format from a more realistic dataset format

Linear Regression – Hands on PySpark



```
from pyspark.ml.regression import LinearRegression
```

Load

```
# Load training data  
training = spark.read.format("libsvm").load("./sample_linear_regression_dat
```

Settings

```
lr = LinearRegression(featuresCol='features',  
                       labelCol='label',  
                       predictionCol='prediction')
```

Train

```
# Fit the model  
lrModel = lr.fit(training)
```

Evaluate

```
# Summarize the model over the training set and print out some metrics  
trainingSummary = lrModel.summary  
  
trainingSummary.residuals.show()  
print("RMSE: {}".format(trainingSummary.rootMeanSquaredError))  
print("r2: {}".format(trainingSummary.r2))
```

Train/Test Splits



```
# Pass in the split between training/test as a list.  
# No correct, but generally 70/30 or 60/40 splits are used.  
# Depending on how much data you have and how unbalanced it is.  
train_data, test_data = all_data.randomSplit([0.7, 0.3])
```

label	features
-28.046018037776633	(10, [0, 1, 2, 3, 4, 5, ...
-26.736207182601724	(10, [0, 1, 2, 3, 4, 5, ...
-19.16829262296376	(10, [0, 1, 2, 3, 4, 5, ...
-18.27521356600463	(10, [0, 1, 2, 3, 4, 5, ...
-17.494200356883344	(10, [0, 1, 2, 3, 4, 5, ...
-17.32672073267595	(10, [0, 1, 2, 3, 4, 5, ...
-16.71909683360509	(10, [0, 1, 2, 3, 4, 5, ...
-16.2614302754	(10, [0, 1, 2, 3, 4, 5, ...
-16.15134935127	(10, [0, 1, 2, 3, 4, 5, ...
-15.34887115537	(10, [0, 1, 2, 3, 4, 5, ...
-13.867087895158768	(10, [0, 1, 2, 3, 4, 5, ...
-13.15333560636553	(10, [0, 1, 2, 3, 4, 5, ...
-12.977848725392104	(10, [0, 1, 2, 3, 4, 5, ...
-12.92222310337042	(10, [0, 1, 2, 3, 4, 5, ...
-12.558575788856189	(10, [0, 1, 2, 3, 4, 5, ...
-12.500773785355054	(10, [0, 1, 2, 3, 4, 5, ...
-12.491442077546413	(10, [0, 1, 2, 3, 4, 5, ...
-12.130353212287929	(10, [0, 1, 2, 3, 4, 5, ...

70%

label	features
-28.046018037776633	(10, [0, 1, 2, 3, 4, 5, ...
-26.73620718260	(10, [0, 1, 2, 3, 4, 5, ...
-19.1682926229	(10, [0, 1, 2, 3, 4, 5, ...
-18.2752135660	(10, [0, 1, 2, 3, 4, 5, ...
-17.494200356883344	(10, [0, 1, 2, 3, 4, 5, ...
-17.32672073267595	(10, [0, 1, 2, 3, 4, 5, ...

30%

Train/Test Splits and Unlabeled Dataset



```
test_data.show()
```

label	features
-28.046018037776633	(10,[0,1,2,3,4,5,...
-26.736207182601724	(10,[0,1,2,3,4,5,...
-19.16829262296376	(10,[0,1,2,3,4,5,...
-18.27521356600463	(10,[0,1,2,3,4,5,...
-17.494200356883344	(10,[0,1,2,3,4,5,...
-17.32672073267595	(10,[0,1,2,3,4,5,...
-16.71909683360509	(10,[0,1,2,3,4,5,...
-16.26143027545273	(10,[0,1,2,3,4,5,...
-16.151349351277112	(10,[0,1,2,3,4,5,...
-15.348871155379253	(10,[0,1,2,3,4,5,...
-13.867087895158768	(10,[0,1,2,3,4,5,...
-13.15333560636553	(10,[0,1,2,3,4,5,...
-12.977848725392104	(10,[0,1,2,3,4,5,...
-12.92222310337042	(10,[0,1,2,3,4,5,...
-12.558575788856189	(10,[0,1,2,3,4,5,...
-12.500773785355054	(10,[0,1,2,3,4,5,...
-12.491442077546413	(10,[0,1,2,3,4,5,...
-12.130353212287929	(10,[0,1,2,3,4,5,...
-11.857350365429426	(10,[0,1,2,3,4,5,...
-11.640549677888826	(10,[0,1,2,3,4,5,...

only showing top 20 rows

```
unlabeled_data = test_data.select('features')
```

```
unlabeled_data.show()
```

features
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...
(10,[0,1,2,3,4,5,...

only showing top 20 rows

Testset
vs
Unlabeled data

Evaluation



```
correct_model = lr.fit(train_data)
```

Fit on the
trainingset and
evaluate on the
testset

```
test_results = correct_model.evaluate(test_data)
```

```
test_results.residuals.show()  
print("RMSE: {}".format(test_results.rootMeanSquaredError))
```

Evaluation



- Any model that attempts to predict continuous values (unlike categorical values, which is classification)

- Mean Absolute Error

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Mean Squared Error

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Root Mean Square Error

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- R Squared Values

- By itself, R^2 , won't tell you the “whole story”.
- In a basic sense it is a measure of how much variance your model accounts for.
- Between 0-1 (0% to 100%)
- There are also different ways of obtaining R^2 , such as adjusted R squared.
- Can enhance your understanding of a model, or help you compare models, but it shouldn't be your sole source of evaluating a model

Prediction



```
predictions = correct_model.transform(unlabeled_data)
```

```
predictions.show()
```

features	prediction
(10,[0,1,2,3,4,5,...	-1.8291581327047166
(10,[0,1,2,3,4,5,...	-4.022135913269196
(10,[0,1,2,3,4,5,...	-3.826057190201783
(10,[0,1,2,3,4,5,...	-0.5043788046887584
(10,[0,1,2,3,4,5,...	-2.9138888261945266
(10,[0,1,2,3,4,5,...	-1.8111125370265664
(10,[0,1,2,3,4,5,...	-0.8661311314744677
(10,[0,1,2,3,4,5,...	1.5921757023961487
(10,[0,1,2,3,4,5,...	0.867715966812739

On unlabeled data to carry out the predictions



SparkML

Classification Algorithm Example

Classification Problems



- So far we've only seen regression problems where we try to predict a continuous value.
- Not all labels are continuous, sometimes you need to predict categories, this is known as classification.

Classification Problems



- We want to learn about Logistic Regression as a method for Classification.
- Some examples of classification problems:
 - Spam versus “Ham” emails
 - Loan Default (yes/no)
 - Disease Diagnosis
- Above were all examples of Binary Classification
- After you train a classification model on some training data, you will evaluate your model’s performance on some test data.

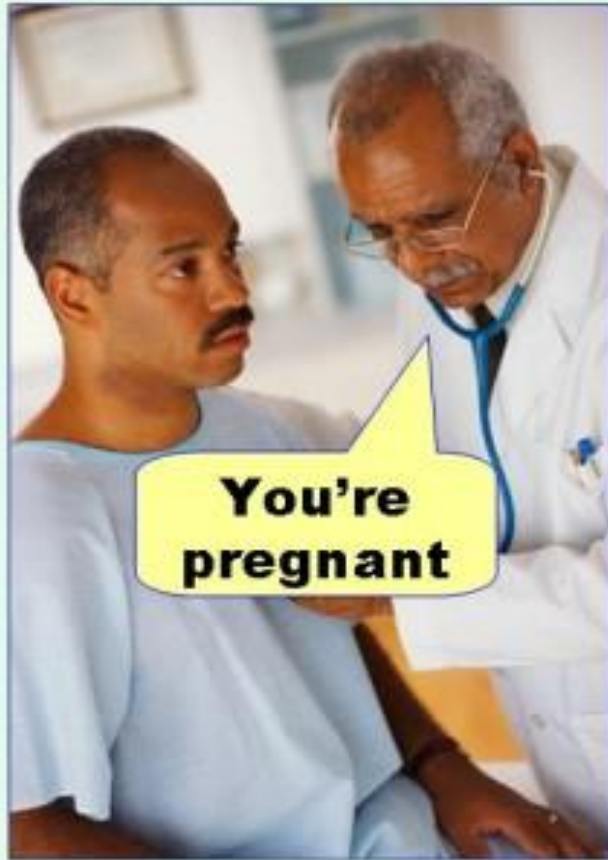
Confusion Matrix



		predicted condition	
		prediction positive	prediction negative
true condition	total population		
	condition positive	True Positive (TP)	False Negative (FN) (type II error)
	condition negative	False Positive (FP) (Type I error)	True Negative (TN)

Confusion Matrix

Type I error
(false positive)



Type II error
(false negative)



Confusion Matrix



		predicted condition		
		total population		
			prediction positive	prediction negative
true condition	condition positive	True Positive (TP)	False Negative (FN) (type II error)	Prevalence $= \frac{\Sigma \text{ condition positive}}{\Sigma \text{ total population}}$
	condition negative	False Positive (FP) (Type I error)	True Negative (TN)	True Positive Rate (TPR), Sensitivity, Recall, Probability of Detection $= \frac{\Sigma \text{ TP}}{\Sigma \text{ condition positive}}$
				False Positive Rate (FPR), Fall-out, Probability of False Alarm $= \frac{\Sigma \text{ FP}}{\Sigma \text{ condition negative}}$
		Positive Predictive Value (PPV), Precision $= \frac{\Sigma \text{ TP}}{\Sigma \text{ prediction positive}}$	False Omission Rate (FOR) $= \frac{\Sigma \text{ FN}}{\Sigma \text{ prediction negative}}$	Positive Likelihood Ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$
		False Discovery Rate (FDR) $= \frac{\Sigma \text{ FP}}{\Sigma \text{ prediction positive}}$	Negative Predictive Value (NPV) $= \frac{\Sigma \text{ TN}}{\Sigma \text{ prediction negative}}$	Negative Likelihood Ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
		Accuracy $= \frac{\Sigma \text{ TP} + \Sigma \text{ TN}}{\Sigma \text{ total population}}$		

Model Evaluation



- The main point to remember with the confusion matrix and the various calculated metrics is that they are all fundamentally ways of comparing the predicted values versus the true values.
- What constitutes “good” metrics, will really depend on the specific situation!

Logistic Regression – Hands on PySpark



```
from pyspark.ml.regression import LogisticRegression
```

Load

```
# Load training data
training = spark.read.format("libsvm").load("./sample_linear_regression_data")
```

Settings

```
lr = LogisticRegression()
```

Train

```
# Fit the model
lrModel = lr.fit(training)

trainingSummary = lrModel.summary
```

Class prediction
on the training
dataset itself

```
trainingSummary.predictions.show()
```

label	features	rawPrediction	probability	prediction
0.0	(692,[127,128,129...]	[19.8534775947478...]	[0.99999999761359...]	0.0
1.0	(692,[158,159,160...]	[-20.377398194908...]	[1.41321555111056...]	1.0
1.0	(692,[124,125,126...]	[-27.401459284891...]	[1.25804865126979...]	1.0

Evaluation



```
# Usually would do this on a separate test set!  
predictionAndLabels = lrModel.evaluate(training)
```

Valid only for the
example

```
predictionAndLabels.predictions.show()
```

label	features	rawPrediction	probability	prediction
0.0	(692,[127,128,129...	[19.8534775947479...	[0.99999999761359...	0.0
1.0	(692,[158,159,160...	[-20.377398194909...	[1.41321555110962...	1.0
1.0	(692,[124,125,126...	[-27.401459284891...	[1.25804865127002...	1.0

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

```
evaluator = MulticlassClassificationEvaluator(predictionCol='prediction',  
                                              labelCol='label',  
                                              metricName='accuracy')
```

```
acc = evaluator.evaluate(predictionAndLabels)
```



Hands on... Spark ML

Exercise #2: How many crew members a ship will require?

How many crew members a ship will require?



- Your job is to create a regression model that will help predict how many crew members will be needed for future ships.
 - In other words, use the features you think will be useful to predict the value in the Crew column.
 - The client also mentioned that they have found that particular cruise lines will differ in acceptable crew counts, so it is most likely an important feature to include in your analysis!
 - The cruise line value is a string however!
- Ship Name
 - Cruise Line
 - Age (as of 2013)
 - Tonnage (1000s of tons)
 - passengers (100s)
 - Length (100s of feet)
 - Cabins (100s)
 - Passenger Density
 - Crew (100s)



SparkML

From DataFrame to LibSVM format

From DataFrame to LibSVM format



- From CSV ... to DataFrame

```
# Use Spark to read in the Ecommerce Customers csv file.
data = spark.read.csv("Ecommerce_Customers.csv",
                      inferSchema=True,
                      header=True)
```

```
data.printSchema()
```

```
root
|-- Email: string (nullable = true)
|-- Address: string (nullable = true)
|-- Avatar: string (nullable = true)
|-- Avg Session Length: double (nullable = true)
|-- Time on App: double (nullable = true)
|-- Time on Website: double (nullable = true)
|-- Length of Membership: double (nullable = true)
|-- Yearly Amount Spent: double (nullable = true)
```

From DataFrame to LibSVM format



- From DataFrame
- ... to LibSVM

```
# Import VectorAssembler and Vectors
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
```

```
assembler = VectorAssembler(
    inputCols=["Avg Session Length", "Time on App",
               "Time on Website", 'Length of Membership'],
    outputCol="features")
```

```
output = assembler.transform(data)
```

```
output.select(["features", "Yearly Amount Spent"]).show()
```

```
+-----+-----+
|          features|Yearly Amount Spent|
+-----+-----+
|[34.4972677251122...| 587.9510539684005|
|[31.9262720263601...| 392.2049334443264|
|[33.0009147556426...| 487.54750486747207|
|[34.3055566297555...| 581.8523440352177|
```

How many crew members a ship will require?



- Your job is to create a regression model that will help predict how many crew members will be needed for future ships.
 - In other words, use the features you think will be useful to predict the value in the Crew column.
- Ship Name
 - Cruise Line
 - Age (as of 2013)
 - Tonnage (1000s of tons)
 - passengers (100s)
 - Length (100s of feet)
 - Cabins (100s)
 - Passenger Density
 - Crew (100s)

Data: ***cruise_ship_info.csv***

20 min



SparkML

From String to Index

How many crew members a ship will require?



- Your job is to create a regression model that will help predict how many crew members will be needed for future ships.
- In other words, use the features you think will be useful to predict the value in the Crew column.

Data: ***cruise_ship_info.csv***

- Ship Name
- Cruise Line
- Age (as of 2013)
- Tonnage (1000s of tons)
- passengers (100s)
- Length (100s of feet)
- Cabins (100s)
- Passenger Density
- Crew (100s)

5 min

LibSVM Data Format



- Let's suppose we have a string column in a Dataframe that is a feature useful for our Machine Learning Model

```
# Load training data
data = spark.read.csv("./cruise_ship_info.csv", inferSchema=True, header=True)
```

```
data.printSchema()
```

```
root
```

```
| -- Ship name: string (nullable = true)
| -- Cruise line: string (nullable = true)
| -- Age: integer (nullable = true)
| -- Tonnage: double (nullable = true)
| -- passengers: double (nullable = true)
| -- length: double (nullable = true)
| -- cabins: double (nullable = true)
| -- passenger_density: double (nullable = true)
| -- crew: double (nullable = true)
```

Suppose that it is
useful feature for
estimating your
dependent variable



LibSVM Data Format



- Index its values by means of StringIndexer
- Put the index values in a new column

```
from pyspark.ml.feature import StringIndexer
```

```
indexer = StringIndexer(inputCol="Cruise_line", outputCol="Cruise_line_index")  
indexed = indexer.fit(data).transform(data)  
indexed.head()
```

```
Row(Ship_name='Journey', Cruise_line='Azamara', Age=6, Tonnage=30.276999999999997, passengers  
=6.94, length=5.94, cabins=3.55, passenger_density=42.64, crew=3.55, Cruise_line_index=16.0)
```

LibSVM Data Format



- To LibSVM by using VectorAssembler

```
assembler = VectorAssembler(  
    inputCols=["Age", "Tonnage",  
               "passengers", 'length', 'cabins', 'passenger_density', 'Cruise_line_index'],  
    outputCol="features")
```

```
output = assembler.transform(indexed)
```

```
output.select('features', 'crew').show()
```

```
+-----+-----+  
|          features | crew |  
+-----+-----+  
|[6.0,30.276999999...| 3.55 |  
|[6.0,30.276999999...| 3.55 |  
|[26.0,47.262,14.8...|  6.7 |  
|[11.0,110.0,29.74...| 19.1 |
```



Hands on... Spark ML

Exercise #2: Predict passengers survived the titanic crash

Predict passengers survived the titanic crash



- We'll Spark Machine Learning to attempt to predict what passengers survived the titanic crash based solely on passenger's features (age, cabin, children, etc...)
- Our data will also have a lot of missing information, so we will need to deal with that as well.
- We'll see some better ways to deal with categorical data through a two-step process.
- We will also show how to use pipelines to set stages and build models that can be easily used again!

How many crew members a ship will require?



- We'll Spark Machine Learning to attempt to predict what passengers survived the titanic crash based solely on passenger's features (age, cabin, children, etc...)

*Data: **titanic.csv***

20 min

OneHotEncoder vs StringIndexer



- As you probably already know, the categorical value assignment can be done using ***StringIndexer*** with SparkML
- ***StringIndexer*** encodes a string column of labels to a column of label indices. The indices are in $[0, \text{numLabels})$, ordered by label frequencies, so the most frequent label gets index 0. The unseen labels will be put at index ***numLabels*** if user chooses to keep them. If the input column is numeric, we cast it to string and index the string values.

OneHotEncoder vs StringIndexer



- As you probably already know, the categorical value assignment can be done using ***StringIndexer*** with SparkML

id	category
0	a
1	b
2	c
3	a
4	a
5	c

id	category	categoryIndex
0	a	0.0
1	b	2.0
2	c	1.0
3	a	0.0
4	a	0.0
5	c	1.0

OneHotEncoder vs StringIndexer



- As you probably already know, the categorical value assignment can be done using ***StringIndexer*** with SparkML

```
from pyspark.ml.feature import StringIndexer

df = spark.createDataFrame(
    [(0, "a"), (1, "b"), (2, "c"), (3, "a"), (4, "a"), (5, "c")],
    ["id", "category"])

indexer = StringIndexer(inputCol="category", outputCol="categoryIndex")
indexed = indexer.fit(df).transform(df)
indexed.show()
```

OneHotEncoder vs StringIndexer



- Why do you need one hot encoding?
 - Numerical values have their own meaning that may mismatch with the interpretation of our feature
 - Model that we are training may be impacted by the values we are associating to the categorical column

CompanyName	Categoricalvalue	Price
VW	1	20000
Acura	2	10011
Honda	3	50000
Honda	3	10000

- VW > Acura > Honda based on the categorical values.
- Supposing your model internally calculates average, then accordingly we get, $1+3 = 4/2 = 2$. Average of VW and Honda is Acura

OneHotEncoder vs StringIndexer

- *OneHotEncoder* (deprecated Spark 2.3.2)
- Use *OneHotEncoderEstimator* instead
- Give an outlook to the Titanic example

CompanyName	Categoricalvalue	Price
VW	1	20
Acura	2	10011
Honda	3	50000
Honda	3	10000



OneHotEncoding

VW	Acura	Honda	Price
1	0	0	20000
0	1	0	10011
0	0	1	50000
0	0	1	10000



SPARK MLlib

- Pipelines

Pipelines

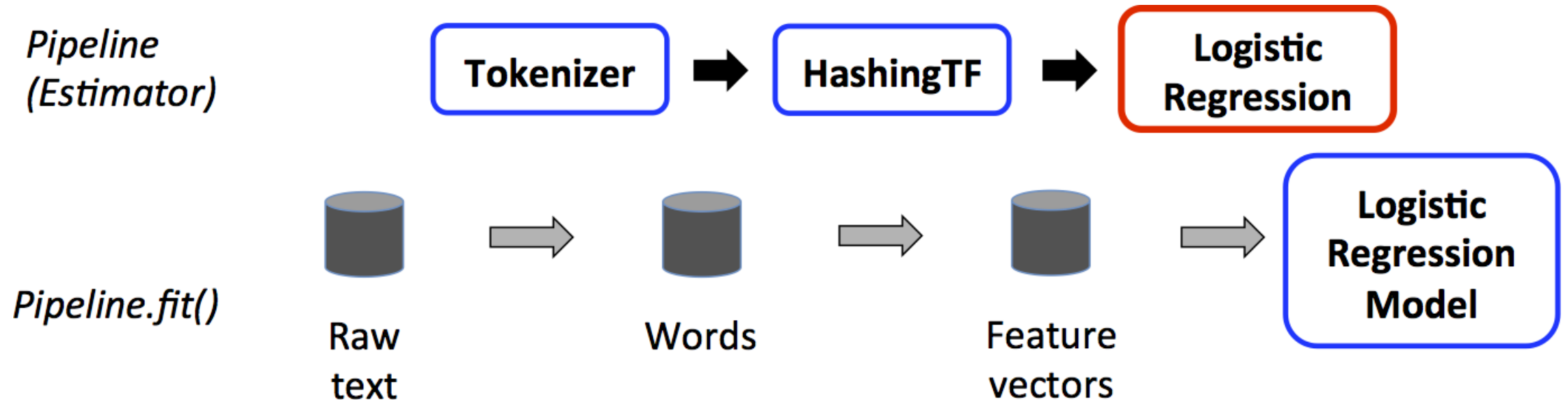


- **DataFrame**: This ML API uses DataFrame from Spark SQL as an ML dataset, which can hold a variety of data types. E.g., a DataFrame could have different columns storing text, feature vectors, true labels, and predictions.
- **Transformer** (transform()): A Transformer is an algorithm which can transform one DataFrame into another DataFrame. E.g., an ML model is a Transformer which transforms a DataFrame with features into a DataFrame with predictions.
- **Estimator** (fit()): An Estimator is an algorithm which can be fit on a DataFrame to produce a Transformer. E.g., a learning algorithm is an Estimator which trains on a DataFrame and produces a model.
- **Pipeline**: A Pipeline chains multiple Transformers and Estimators together to specify an ML workflow.
- **Parameter**: All Transformers and Estimators now share a common API for specifying parameters.

Estimator Pipeline



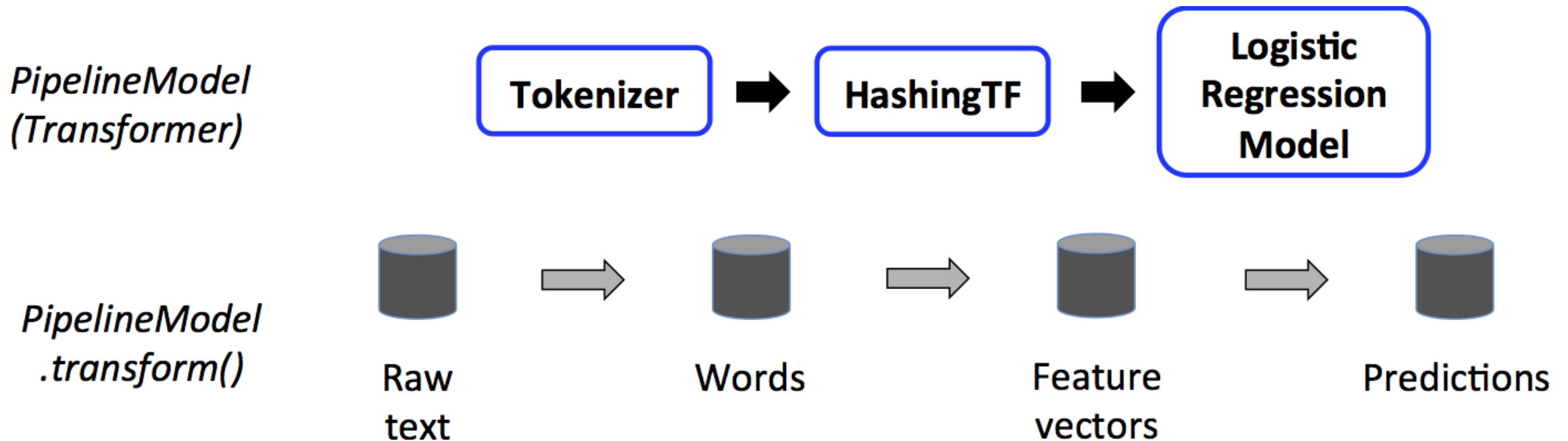
- Split each document's text into words.
- Convert each document's words into a numerical feature vector.
- Learn a prediction model using the feature vectors and labels.



Transformer Pipeline



- Split each document's text into words.
- Convert each document's words into a numerical feature vector.
- Learn a prediction model using the feature vectors and labels.
- Make predictions.





SPARK MLlib

- Clustering

K Means Clustering

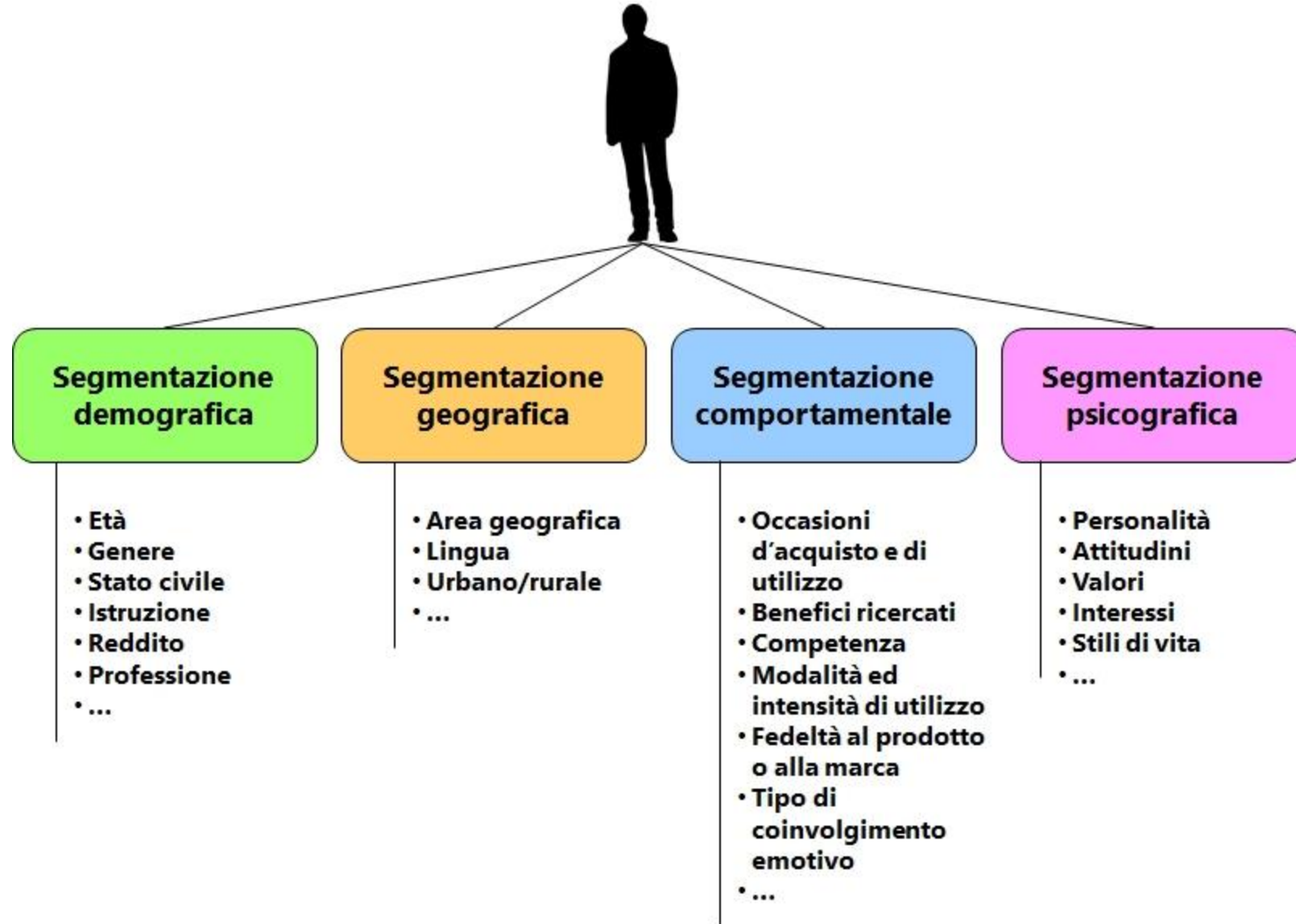


K Means Clustering is an unsupervised learning algorithm that will attempt to group similar clusters together in your data.

So what does a typical clustering problem look like?

- Cluster Similar Documents
- Cluster Customers based on Features
- Market Segmentation
- Identify similar physical groups

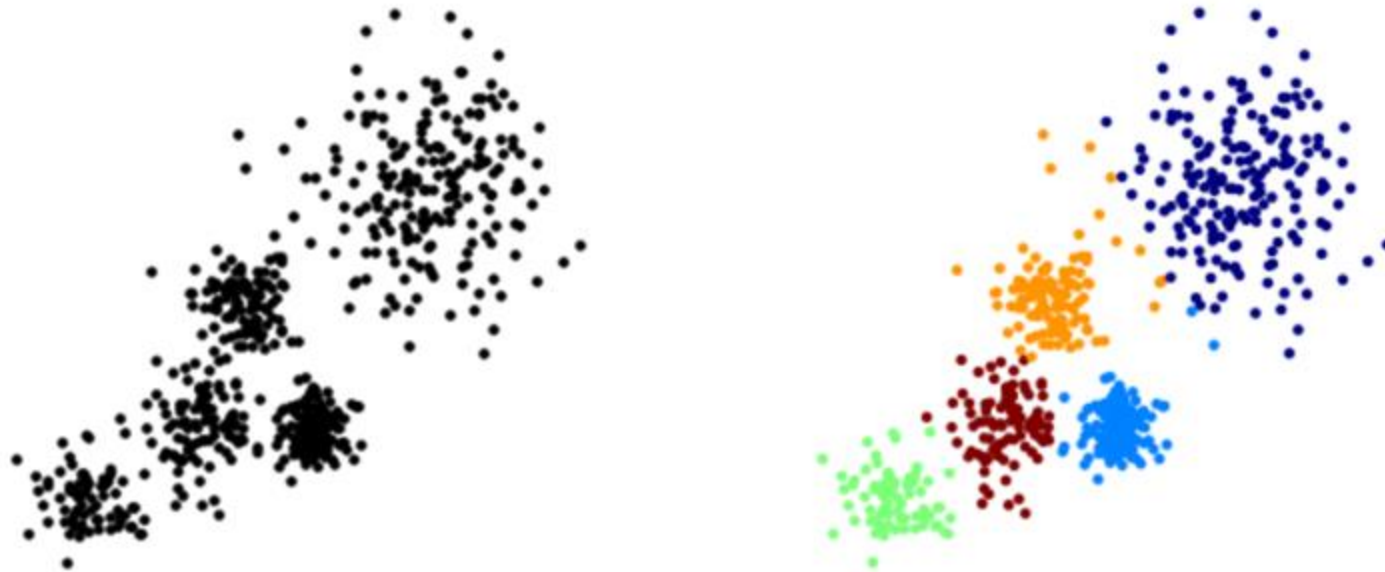
Customer & Market Segmentation



K Means Clustering



- The overall goal is to divide data into distinct groups such that observations within each group are similar



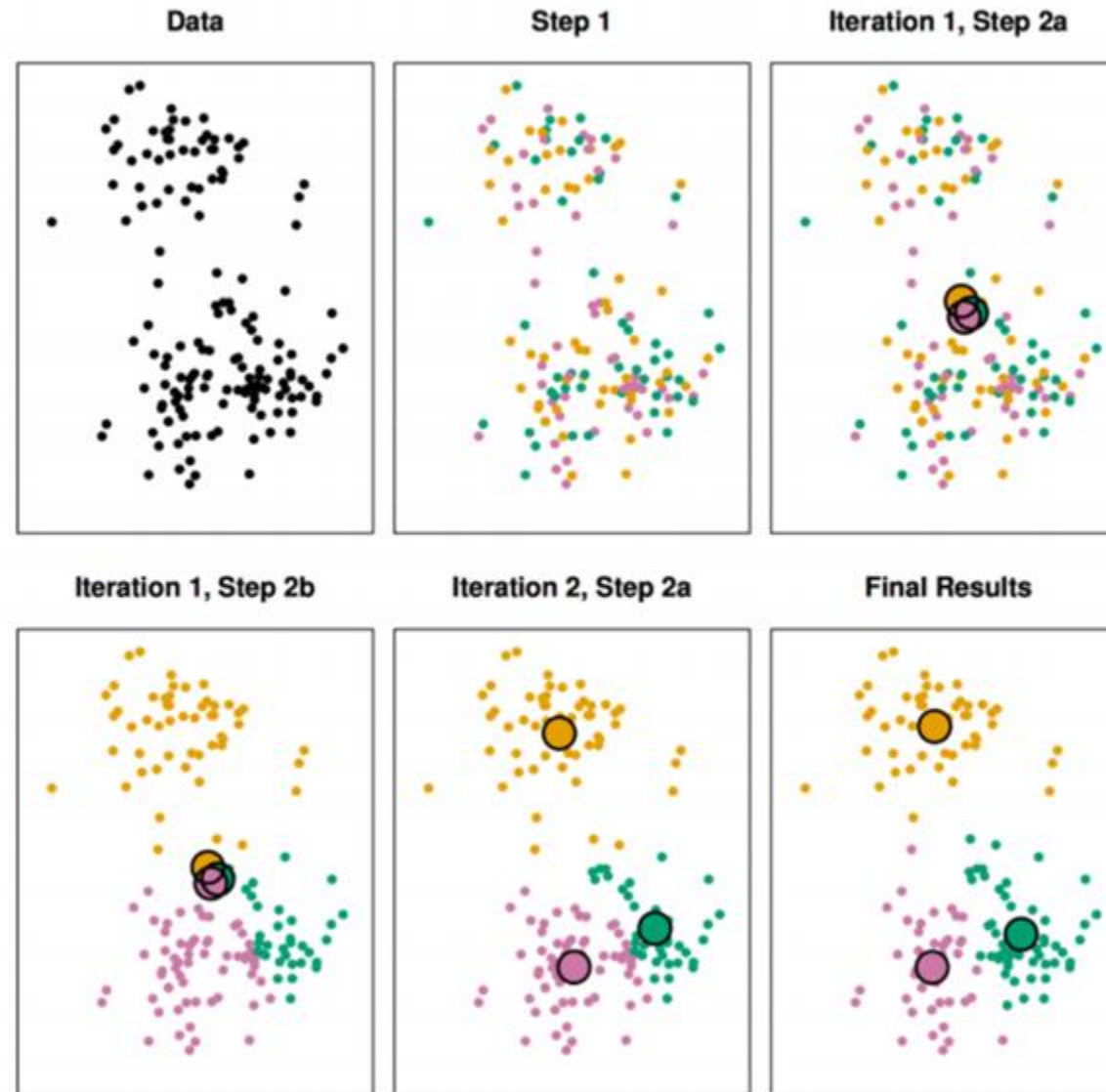
K Means Clustering



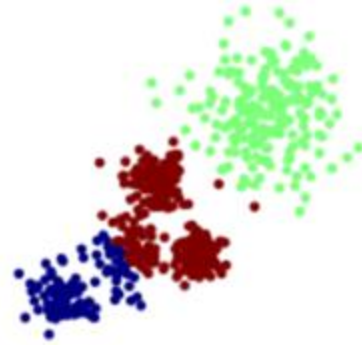
The K Means Algorithm

- Choose a number of Clusters “K”
- Randomly assign each point to a cluster
- Until clusters stop changing, repeat the following:
 - For each cluster, compute the cluster centroid by taking the mean vector of points in the cluster
 - Assign each data point to the cluster for which the centroid is the closest

K Means Clustering



Choosing a K Value



Choosing a K Value



- There is no easy answer for choosing a “best” K value
- One way is the elbow method

First of all, compute the sum of squared error (SSE) for some values of k (for example 2, 4, 6, 8, etc.).

The SSE is defined as the sum of the squared distance between each member of the cluster and its centroid.

Choosing a K Value

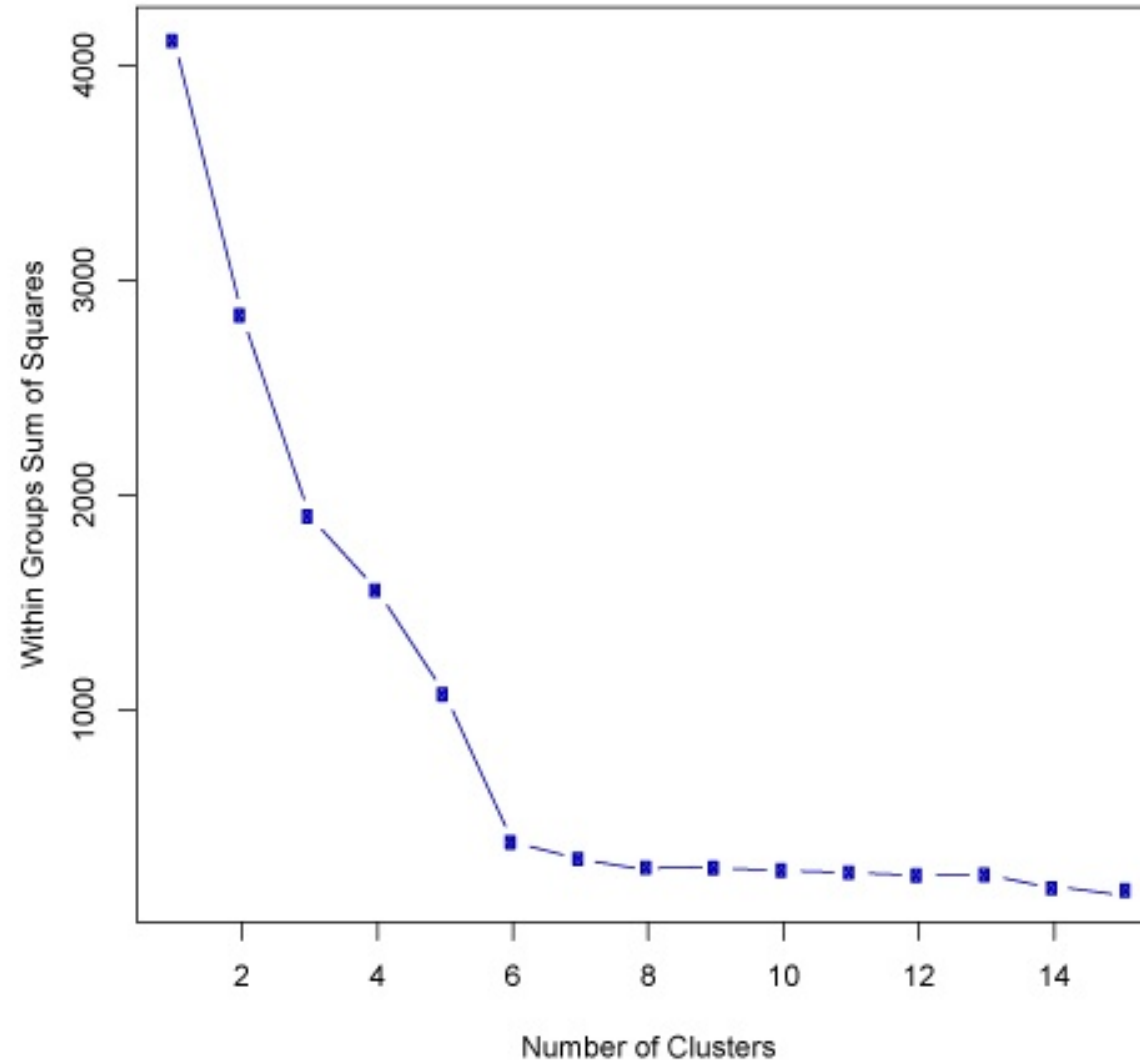


If you plot k against the SSE, you will see that *the error decreases as k gets larger*; this is because when the number of clusters increases, they should be smaller, so distortion is also smaller.

The idea of the elbow method is to choose the k at which the SSE decreases abruptly.

Choosing a K Value

This produces an "elbow effect" in the graph, as you can see in the following picture



K-Means with Spark



KMeans is implemented as an `Estimator` and generates a `KMeansModel` as the base model.

Input Columns

Param name	Type(s)	Default	Description
featuresCol	Vector	"features"	Feature vector

Output Columns

Param name	Type(s)	Default	Description
predictionCol	Int	"prediction"	Predicted cluster center

K-Means with Spark



```
from pyspark.ml.clustering import KMeans

# Loads data.
dataset = spark.read.format("libsvm").load("sample_kmeans_data.txt")

# Trains a k-means model.
kmeans = KMeans().setK(2).setSeed(1)
model = kmeans.fit(dataset)

# Evaluate clustering by computing Within Set Sum of Squared Errors.
wsse = model.computeCost(dataset)
print("Within Set Sum of Squared Errors = " + str(wsse))

# Shows the result
centers = model.clusterCenters()
print("Cluster Centers: ")
for center in centers:
    print(center)

model.transform(dataset)
```

featureCol="features"
by default

Choosing a K Value



- Pyspark by itself doesn't support a plotting mechanism, but you could use `collect()` and then plot the results with matplotlib or other visualization libraries.



Hands on... Spark ML

Exercise #3:

Exercise #1



- It's time for you to go to San Francisco to help out a tech startup!
- They've been recently hacked and need your help finding out about the hackers!
- Luckily their forensic engineers have grabbed valuable data about the hacks, including information like session time, locations, wpm typing speed, etc.

Exercise #1



- 'Session_Connection_Time': How long the session lasted in minutes
- 'Bytes Transferred': Number of MB transferred during session
- 'Kali_Trace_Used': Indicates if the hacker was using Kali Linux
- 'Servers_Corrupted': Number of server corrupted during the attack
- 'Pages_Corrupted': Number of pages illegally accessed
- 'Location': Location attack came from (Probably useless because the hackers used VPNs)
- 'WPM_Typing_Speed': Their estimated typing speed based on session logs.

Exercise #1



- The technology firm has 3 potential hackers that perpetrated the attack.
- They are certain of the first two hackers but they aren't very sure if the third hacker was involved or not.
- They have requested your help!

Exercise #1



- Can you help figure out whether or not the third suspect had anything to do with the attacks, or was it just two hackers?
- One last key fact, the forensic engineer knows that the hackers trade off attacks.
- Meaning they should each have roughly the same amount of attacks.